

Modular Multi-Agent Cognitive Pipeline

A Domain-Agnostic Architecture for Paraconsistent
Knowledge Synthesis with Snap-In Modularity

Jero Smith

Oovo0 Research

March 11, 2026

Version 1.0 — Architectural Specification

Abstract

This paper presents a modular, environment-agnostic pipeline architecture for multi-agent knowledge synthesis. The system decomposes complex queries into morphological matrices, spawns constrained specialist agents in parallel, and consolidates their findings through a three-stage paraconsistent consensus engine that treats contradiction as a productive signal rather than an error condition. The architecture is defined entirely through standardized connector points—typed input/output interfaces, processing loops, and connection schemas—enabling any component to be snapped in, swapped out, or forked without altering the pipeline spine. Deployment is abstracted across single-node, cloud-orchestrated, and peer-to-peer environments through a uniform execution layer. We formalize seven thread execution patterns (atomic, parallel, chained, fusion, meta-orchestration, long-duration, and zero-touch), four connector types (sequential, feedback, longitudinal, and graph-relational), and four branching points that support runtime forking. The result is a pipeline that can be instantiated for any knowledge domain by plugging in domain-specific data sources and agent configurations without modifying the core architecture.

Keywords: multi-agent systems, modular architecture, paraconsistent logic, knowledge synthesis, morphological analysis, snap-in modularity, domain-agnostic pipelines

Table of Contents

1. Introduction and Motivation
2. Architectural Overview
3. The Pipeline Spine: Five-Stage Processing Model
4. Standardized Connector Points
5. Thread Execution Patterns
6. Agent Specification and Constraint Model
7. The Paraconsistent Consensus Engine
8. Forking, Branching, and Snap-In Modularity
9. Environment-Agnostic Deployment
10. Interface Schemas and Type Contracts
11. Processing Loops and Feedback Mechanisms
12. Failure Modes and Mitigation
13. Scaling Properties
14. Reference Implementation Guidance
15. Conclusion

1. Introduction and Motivation

Contemporary AI-assisted knowledge synthesis faces a structural limitation: most systems treat the problem as single-agent question-answering. A user poses a query, a model retrieves context, and produces a single-perspective answer. This architecture discards the most valuable signal in complex knowledge work—the productive tension between contradictory expert perspectives.

This paper introduces a modular pipeline architecture that addresses three fundamental gaps in existing multi-agent frameworks:

- **Contradiction as signal:** Standard consensus mechanisms average out disagreement. Our paraconsistent engine preserves contradictions as first-class outputs—cases where two agents disagree may represent the most novel and valuable insights.
- **Domain lock-in:** Existing multi-agent research systems are built for specific domains (scientific literature, code analysis, market research). This architecture defines pure structural abstractions—any domain can be plugged in by implementing a standard connector interface.
- **Environment coupling:** Most pipelines assume a specific runtime (cloud functions, container orchestration, single-process). This architecture abstracts execution into a uniform layer that deploys identically on a single laptop, a cloud cluster, or a peer-to-peer mesh network.

The key innovation is **snap-in modularity**: every component—from data sources to agent strategies to output formatters—connects through typed interfaces. New capabilities are added by implementing an interface and registering in the fork registry, without modifying any existing pipeline code. This is analogous to Unix pipes, USB connectors, or electrical standards: a universal physical interface enabling unlimited functional diversity.

2. Architectural Overview

The architecture comprises three layers, each operating at a different level of abstraction:

2.1 Layer Model

Layer	Responsibility	Components
Pipeline Spine (Structural)	Defines the five-stage processing flow, connector types, and state transitions	Stages 1–5, Connector Interfaces, State Machine, Fork Registry
Execution Layer (Runtime)	Abstracts how stages run across environments	Thread Patterns (T0–Z), Orchestrator, Memory Bus, Environment Adapters
Domain Layer (Application)	Pluggable components that implement the connector interfaces for a specific knowledge domain	Data Source Adapters, Agent Configurations, Output Formatters, Scoring Functions

Table 1: Three-layer architecture. The Pipeline Spine and Execution Layer are domain-agnostic. Only the Domain Layer contains application-specific code.

This separation ensures that the structural architecture (Spine) and runtime mechanics (Execution) remain constant regardless of what knowledge domain is being explored. A team building a medical literature synthesizer and a team building a patent landscape analyzer would share identical Spine and Execution layers, differing only in their Domain Layer plug-ins.

2.2 Information Flow

Data flows through the pipeline in a single primary direction (left-to-right through Stages 1–5) with three types of secondary flow:

- **Feedback loops** (Stage N output feeds back to Stage N-1 for refinement)
- **Longitudinal connections** (cross-session learning, persistent knowledge accumulation)
- **Graph-relational links** (typed relationships between claims, agents, and patterns across stages)

Each flow type is formalized as a **Connector Type** (Section 4) with its own interface contract, enabling components to participate in any flow pattern without awareness of the others.

3. The Pipeline Spine: Five-Stage Processing Model

The pipeline spine is a five-stage state machine. Each stage has a typed input contract, a typed output contract, and a set of valid state transitions. The stages are:

Stage	Name	Input Contract	Output Contract	State Transitions
1	Query Decomposition	Raw query (string) + context metadata	Decomposition object: primary domain, secondary domains, key variables, exploration axes, success criteria	initialized → decomposing → decomposed
2	Matrix Generation	Decomposition object	Morphological matrix: N dimensions × M values per dimension, prioritized cell list	decomposed → matrix_ready
3	Agent Deployment	Matrix cells + agent configurations	Set of agent briefs: problem statement, approach, findings, limitations, claims (each with confidence + vector embedding)	matrix_ready → agents_working → agents_complete
4	Paraconsistent Consolidation	All agent briefs + claim vectors	Consensus package: baseline agreements, State-2 contradictions, emergent insights, cross-domain connections, discovered patterns	agents_complete → consolidating → consolidated
5	Output Synthesis	Consensus package + audience specification	Delivery package: formatted output (JSON/Markdown/HTML/PDF), audience-customized presentation	consolidated → synthesizing → completed

Table 2: Five-stage pipeline spine with typed input/output contracts and state transitions.

3.1 Stage Independence Principle

Each stage operates as an independent processing unit that can be tested, replaced, or parallelized without affecting adjacent stages. The only coupling between stages is the typed contract—as long as a replacement component produces a conforming output object, the downstream stage is unaware of the swap. This is the foundation of snap-in modularity: stages are connected by contracts, not by implementation details.

3.2 Stage Detail: Query Decomposition (Stage 1)

The decomposition engine transforms a natural-language query into a structured decomposition object. This involves identifying the primary knowledge domain, extracting secondary domains that may offer cross-pollination value, isolating key variables and their relationships, defining exploration axes (the independent dimensions along which the query can be investigated), and establishing machine-verifiable success criteria. The decomposition engine is itself a pluggable component—a simple implementation might use a single LLM call with structured output, while an advanced implementation could use a multi-step chain

with domain-specific taxonomies.

3.3 Stage Detail: Matrix Generation (Stage 2)

The morphological matrix (following Zwicky's General Morphological Analysis) creates a systematic combinatorial space from the exploration axes. For a query decomposed into N axes with M_i values per axis, the matrix contains $\prod M_i$ cells. Each cell represents a unique combination of axis values—a specific perspective to investigate. The matrix generator prioritizes cells using pluggable scoring functions (diversity, novelty, feasibility, information gain) and selects the top K cells for agent assignment.

3.4 Stage Detail: Agent Deployment (Stage 3)

For each prioritized matrix cell, the pipeline spawns a constrained specialist agent (see Section 6). Agents operate independently and in parallel, each producing a structured research brief. The critical design choice is **agent independence**—agents do not communicate with each other during Stage 3. This prevents premature consensus and preserves the diversity of perspectives that makes Stage 4 productive. Each brief includes extracted claims with confidence levels and vector embeddings, enabling semantic similarity comparison in the consolidation stage.

3.5 Stage Detail: Paraconsistent Consolidation (Stage 4)

This is the architectural core of the system and is detailed fully in Section 7. In summary, consolidation proceeds through three sub-stages: (a) baseline extraction identifies claims with strong multi-agent agreement, (b) State-2 localization identifies claims with significant support AND significant contradiction—these are flagged as paraconsistent rather than discarded, and (c) emergent synthesis detects patterns visible only when multiple agent perspectives are held simultaneously, including cross-domain connections and higher-order structures.

3.6 Stage Detail: Output Synthesis (Stage 5)

The delivery layer transforms the consensus package into audience-specific output formats. An audience specification object defines the target reader's expertise level, preferred format, required depth, and which categories of insight (baseline, contradiction, emergent) to emphasize. Output formatters are pluggable—the same consensus package can produce an academic paper, a policy brief, a technical specification, or an executive summary by swapping the formatter.

4. Standardized Connector Points

The connector system is the mechanism that enables snap-in modularity. Every point where two components interact is mediated by a typed connector interface. We define four connector types, each serving a different flow pattern:

Type	ID	Flow Pattern	Interface Contract	Use Cases
Sequential	A	Stage N $\rightarrow$ Stage N+1 (forward-only)	$\text{output}(\text{StageN}) \subseteq \text{input}(\text{StageN+1})$ Strict type conformance. No backward reference.	Primary pipeline flow. Each stage consumes its predecessor's output.
Feedback	B	Stage N $\rightarrow$ Stage N-K (backward loop)	<code>feedback_signal: { source_stage: int, target_stage: int, refinement_request: object, iteration_count: int, max_iterations: int }</code>	Iterative refinement. Agent re-runs after consolidation feedback. Query re-decomposition.
Longitudinal	C	Session N $\rightarrow$ Session N+1 (cross-session)	<code>knowledge_delta: { new_claims: Claim[], updated_confidences: Map, deprecated_claims: ID[], session_id: string }</code>	Cross-session learning. Persistent knowledge accumulation. Trend detection.
Graph-Relational	D	Any $\leftrightarrow$ Any (non-linear)	<code>connection: { source_id: string, target_id: string, relation_type: enum, confidence: float, context: object }</code>	Claim $\leftrightarrow$ Claim links. Pattern discovery. Contradiction resolution graph traversal.

Table 3: Four connector types. Every inter-component interaction uses exactly one of these.

4.1 Connector Registration Protocol

When a new component is added to the system, it registers its connector requirements in the **Fork Registry**—a declarative manifest listing which connector types it consumes and produces. The registry validates type compatibility at registration time, preventing runtime mismatches. This is analogous to a plugin manifest declaring its API dependencies:

```
component_id: "custom_scorer_v2"
attaches_to: stage_2
consumes: [ConnectorType.A] # receives decomposition
produces: [ConnectorType.A] # emits scored matrix
optional_consumes: [ConnectorType.C] # can use cross-session data
config_schema: { weights: float[4], threshold: float }
```

4.2 Relation Types (Type D Connectors)

Graph-relational connectors support eight typed relationships between any two objects in the system (claims, agents, patterns, sessions):

Relation	Semantics	Symmetry	Tier
supports	Claim A provides evidence for Claim B	Asymmetric	1 (direct)

Relation	Semantics	Symmetry	Tier
contradicts	Claim A and Claim B cannot both hold in the same context	Symmetric	1 (direct)
complements	Claims A and B cover different aspects of the same phenomenon	Symmetric	1 (direct)
enables	Claim A is a prerequisite for Claim B	Asymmetric	2 (causal)
bridges_domains	Connection spans two different knowledge domains	Symmetric	2 (causal)
generalizes	Claim A is a higher-abstraction form of Claim B	Asymmetric	3 (meta)
instantiates	Claim A is a concrete example of pattern B	Asymmetric	3 (meta)
co_evolves	Claims A and B change together over time (longitudinal)	Symmetric	3 (meta)

Table 4: Eight relation types organized into three tiers of abstraction.

5. Thread Execution Patterns

Thread patterns define *how* pipeline stages execute at runtime. They are the Execution Layer's vocabulary for composing work. Seven patterns form a complete set, from minimal human oversight to full autonomy:

Pattern	ID	Structure	Human Involvement	Best For
Atomic	T0	Single agent executes one task. Human reviews before next step.	Review after every action	Unit testing components. High-stakes single tasks.
Parallel	P	N agents execute concurrently. Partitioned (each gets unique work) or Redundant (same work, different approaches).	Review after all complete	Stage 3 agent deployment. Redundant = fusion input.
Chained	C	Phased sequential execution. Phase N output = Phase N+1 input. Human checkpoint between phases.	Checkpoint between phases	The 5-stage pipeline spine. Linear dependency chains.
Fusion	F	Parallel outputs merged via: (a) Best-of-N selection, (b) Ensemble merge, or (c) Debate + adjudication.	Review fused output	Stage 4 consolidation. Multi-perspective synthesis.
Meta-Orchestration	B	Orchestrator spawns and manages sub-agents. Compositions: Plan→Build, Scout→Implement→Review, or Full Team.	Review orchestrator output. Sub-agent oversight is delegated.	Complex multi-step tasks. Self-organizing agent teams.
Long-Duration	L	Extended autonomous operation with self-verification checkpoints and stop hooks.	Periodic check-ins. Stop hooks for safety.	Cross-session learning. Longitudinal analysis.
Zero-Touch	Z	Full autonomy. No human review. Verification delegated to automated tests/policies.	None (post-hoc audit only)	CI/CD pipelines. Routine re-runs of validated configurations.

Table 5: Seven thread execution patterns. Patterns compose hierarchically—a B-thread can contain P-threads, which contain T0-threads.

5.1 Thread Composition

Threads compose hierarchically. The default pipeline execution is a **C-thread** (chained) containing: a T0-thread for Stage 1 decomposition, a B-thread for Stage 2 matrix generation (the orchestrator may spawn sub-agents for scoring), a P-thread for Stage 3 (parallel agent deployment), an F-thread for Stage 4 (fusion/consolidation), and a T0-thread for Stage 5 (output synthesis). This composition can be overridden at any level—for example, replacing the Stage 4 F-thread with a B-thread that uses debate+adjudication internally.

5.2 Thread Evolution Path

Systems mature along four axes: **more threads** (broader parallelism), **longer threads** (more stages per execution), **thicker threads** (richer context per agent), and **fewer human checkpoints** (progressive trust

delegation from T0 toward Z). Each axis can be advanced independently, allowing teams to increase autonomy in well-validated stages while maintaining human oversight in novel or high-risk components.

6. Agent Specification and Constraint Model

Agents are the pipeline's primary processing units. Each agent is defined by a **constraint specification**—a bounded set of parameters that determines its expertise, methodology, resource limits, and output format. The constraint model serves two purposes: it ensures agent diversity (each has a distinct perspective) and it enables reproducibility (the same constraint spec always produces the same behavioral envelope).

6.1 Agent Constraint Schema

Parameter	Type	Description	Example Values
agent_id	UUID	Unique identifier	Auto-generated
title	string	Human-readable role name	"Cross-Domain Analyst"
expertise_domain	string[]	Knowledge domains the agent draws from	["graph theory", "systems dynamics"]
methodology_toolkit	string[]	Allowed analytical methods	["comparative analysis", "causal inference"]
data_sources	string[]	Permitted data source types	["vector_store", "knowledge_graph"]
scope_boundary	string	What the agent must NOT investigate	"Exclude implementation details; focus on theoretical frameworks"
output_word_limit	int	Maximum output length	3000
timeline_hours	float	Maximum processing time	2.0
confidence_threshold	float	Minimum confidence for claim inclusion	0.6
peer_review_group	string[]	Agent IDs that review this agent's output	[agent_id_1, agent_id_2]

Table 6: Agent constraint schema. All fields are required. Domain Layer plug-ins provide the actual values; the Pipeline Spine enforces the structure.

6.2 Agent Output Schema

Every agent produces output conforming to a standardized **Research Brief** structure. This ensures that Stage 4 (consolidation) can process agent outputs uniformly regardless of what the agent investigated or how it investigated it:

- **Problem Statement:** What question did this agent investigate?
- **Approach:** What methodology was applied? Which data sources were consulted?
- **Findings:** What was discovered? Structured as a list of claims.
- **Limitations:** What could not be determined? What assumptions were made?
- **Implications:** What do these findings mean for the broader query?
- **Claims:** Extracted atomic assertions, each with: text, confidence (0.0–1.0), evidence citations, and a vector embedding for semantic comparison.

6.3 Agent Independence Invariant

During Stage 3 execution, agents must not have access to other agents' outputs. This invariant prevents *premature consensus*—the tendency for later agents to converge on early agents' conclusions rather than exploring genuinely distinct perspectives. The Execution Layer enforces this by isolating agent contexts (separate memory spaces, no shared state during Stage 3). Agent outputs are only brought together in Stage 4.

7. The Paraconsistent Consensus Engine

The consensus engine is the novel core of the architecture. Unlike standard aggregation methods (majority vote, weighted average, Delphi), this engine operates on a three-valued logic system where contradiction is a valid and productive state:

State	Symbol	Name	Condition	Semantics
0	∅	Rejected	Agreement < 30% across agents	No meaningful support. Claim is noise or error.
1	✓	Accepted	Agreement ≥ 60% across agents	Strong convergence. High-confidence baseline.
2	‡	Paraconsistent	Support 30–70% AND active contradiction from other agents	Both support and contradiction are meaningful. Hold in suspension.

Table 7: Three-valued logic for claim classification. State 2 is the key innovation—contradictions are preserved rather than resolved.

7.1 Sub-Stage A: Baseline Extraction

All claims from all agents are collected and semantically compared using vector similarity (cosine distance on embeddings). Claims with similarity above a configurable threshold (default: 0.85) are grouped into **claim clusters**. Each cluster is scored by its **convergence ratio**: the fraction of agents whose output contains a claim in the cluster. Clusters with convergence ≥ 0.60 are classified as **State 1 (Accepted)** baseline claims.

7.2 Sub-Stage B: State-2 Localization

This sub-stage identifies the most valuable claims—those with meaningful support AND meaningful contradiction. A claim cluster is classified as **State 2 (Paraconsistent)** when:

- Support ratio is between 0.30 and 0.70 (neither consensus nor noise)
- At least one other claim cluster has a *contradicts* relation (Type D connector) to it
- The contradicting cluster also has support ratio ≥ 0.30

State-2 claims are annotated with their contradiction context—which agents support, which contradict, and what conditions would make each side true. The engine applies paraconsistent logic: rather than selecting one side, it records **both true in context** with the respective context conditions. This preserves the information content of the disagreement rather than destroying it through forced resolution.

7.3 Sub-Stage C: Emergent Synthesis

The final sub-stage searches for **higher-order patterns** that are visible only when multiple agent perspectives are held simultaneously. The engine scans the full claim graph (all State-1 and State-2 claims with their Type-D connections) for four classes of emergent structure:

- **Feedback loops:** Circular claim chains (A supports B supports C supports A) that indicate self-reinforcing dynamics.

- **Phase transitions:** State-2 clusters where the context condition that determines which side is true represents a threshold or tipping point.
- **Attractors:** Claim clusters that are supported by agents from many different expertise domains, suggesting a universal or fundamental principle.
- **Bridges:** Claims that connect two otherwise disconnected domain subgraphs, suggesting novel cross-domain transfer opportunities.

8. Forking, Branching, and Snap-In Modularity

The architecture supports four formal branching points where the pipeline can fork into alternative processing paths. Each branching point is defined by a **fork condition** (what triggers the fork), a **fork connector** (how the fork receives and returns data), and a **reconvergence point** (where fork results merge back into the spine).

8.1 Branching Points

Branch	Location	Fork Condition	Example Forks
BP-1	Between Stage 1 → 2	Query complexity assessment. Simple queries skip matrix generation entirely.	Direct-answer fork (simple queries). Multi-domain fork (split by domain). Iterative-deepening fork.
BP-2	Between Stage 2 → 3	Agent deployment strategy. Different strategies for different matrix regions.	Expert-panel fork (few deep agents). Swarm fork (many shallow agents). Hybrid fork (deep + shallow).
BP-3	Within Stage 4	Contradiction resolution strategy. Different approaches for different claim types.	Voting fork (majority wins). Paraconsistent fork (hold both). Debate fork (agents argue).
BP-4	Between Stage 4 → 5	Output format selection. Different audiences need different presentations.	Academic fork (paper format). Brief fork (executive summary). Interactive fork (explorable).

Table 8: Four branching points with example forks. Any fork can be added by implementing the connector interface and registering in the Fork Registry.

8.2 The Fork Registry

The Fork Registry is a declarative manifest that catalogs all available forks and their connector requirements. At pipeline initialization, the runtime reads the registry and wires the active configuration. Adding a new fork requires three steps: (1) implement the component conforming to the appropriate connector interface, (2) write a registry entry specifying which branching point it attaches to, what conditions trigger it, and what connector types it consumes/produces, and (3) register it. No existing code is modified.

8.3 Fork Composition: The Spine+Fork Model

The pipeline operates on a **spine+fork** topology. The spine is the five-stage sequential flow. Forks branch off from specific points on the spine, perform specialized processing, and reconverge. Multiple forks can be active simultaneously at the same branching point (the router dispatches different items to different forks based on configurable routing rules). Forks can themselves contain sub-forks, creating a recursive tree of processing paths—but all paths eventually reconverge at the next spine stage, ensuring the pipeline always produces a unified output.

8.4 Micro-Loops Within Forks

Any fork can internally implement a **micro-loop**—a tight iterative cycle for brainstorming, refinement, or validation. The micro-loop uses Type B (feedback) connectors internally while consuming and producing Type A (sequential) connectors externally. This allows forks to iterate without the spine being aware of the internal iteration count. Micro-loops are bounded by the agent's `timeline_hours` constraint to prevent infinite cycling.

9. Environment-Agnostic Deployment

The architecture defines four deployment abstractions that allow the same pipeline configuration to run in any environment without modification:

Abstraction	Interface	Single-Node Impl.	Cloud Impl.	P2P Impl.
Compute Provider	<code>spawn_agent(spec)</code> <code>get_result(id)</code> <code>cancel(id)</code>	Local process/thread pool. Agents share CPU cores.	Serverless functions or container instances. Auto-scaling.	Peer nodes claim agent tasks via DHT. BitNet-class local inference.
State Store	<code>put(key, value)</code> <code>get(key)</code> <code>query(filter)</code> <code>subscribe(pattern)</code>	SQLite or in-memory. Single process state.	PostgreSQL/Redis. Managed services with replicas.	Distributed hash table (DHT) with CRDT-based merging.
Vector Store	<code>upsert(id, embedding)</code> <code>query(embedding, k)</code> <code>delete(id)</code>	ChromaDB or FAISS. Local index files.	Pinecone, Weaviate, or managed pgvector.	Federated vector index across peers. Each peer holds a partition.
Message Bus	<code>publish(topic, msg)</code> <code>subscribe(topic, fn)</code> <code>request_reply(topic, msg, timeout)</code>	In-process event emitter. Direct function calls.	Kafka, NATS, or cloud pub/sub. Durable messaging.	Libp2p GossipSub. Topic-based pub/sub over mesh network.

Table 9: Four deployment abstractions with implementations for three environments. The pipeline code calls only the interface methods—the implementation is injected at startup.

9.1 Environment Adapter Pattern

At initialization, the pipeline receives an **EnvironmentAdapter** object that provides concrete implementations for all four abstractions. The pipeline code never references specific databases, message brokers, or compute providers directly. Switching from single-node to cloud deployment requires only swapping the adapter—no pipeline code changes. This also enables hybrid deployments: a team might use cloud compute for Stage 3 (agent deployment benefits from parallelism) while running everything else locally.

9.2 P2P-Specific Considerations

Peer-to-peer deployment introduces unique challenges: node availability is non-deterministic, network partitions are expected, and compute capability varies across peers. The P2P adapter handles these through: (a) redundant agent assignment (each matrix cell is assigned to 2–3 peers; first completion wins), (b) CRDT-based state merging (eventually consistent convergence even under partitions), and (c) capability-aware task routing (heavier agents routed to more capable nodes). The pipeline's contract-based design means these P2P concerns are entirely contained within the adapter—the Pipeline Spine is unaware of them.

10. Interface Schemas and Type Contracts

This section formalizes the six core interface schemas that define the system's type contracts. Any component that implements these schemas is guaranteed to interoperate with the pipeline.

10.1 Query Input Specification

Field	Type	Description
query_text	string	Natural language research query
context	object?	Optional context (prior session refs, domain hints)
constraints	object?	Resource limits (max agents, max time, budget)
output_preferences	object?	Desired output format and audience

10.2 Decomposition Output

Field	Type	Description
session_id	UUID	Pipeline session identifier
primary_domain	string	Central knowledge domain
secondary_domains	string[]	Cross-pollination domains
key_variables	Variable[]	Named variables with type and range
exploration_axes	Axis[]	Independent investigation dimensions
success_criteria	Criterion[]	Machine-verifiable completion conditions

10.3 Agent Instantiation Template

Field	Type	Description
agent_spec	AgentConstraint	Full constraint schema (Table 6)
matrix_cell	CellRef	Which matrix cell this agent covers
data_source_config	DataSourceConfig	Connection params for permitted sources
output_schema	BriefSchema	Expected output structure

10.4 Claim Schema

Field	Type	Description
claim_id	UUID	Unique claim identifier
text	string	Atomic assertion text

Field	Type	Description
confidence	float [0,1]	Agent's confidence in this claim
evidence	Citation[]	Supporting evidence references
embedding	float[N]	Vector embedding for semantic comparison
source_agent	UUID	Agent that produced this claim
context_conditions	string?	Conditions under which claim holds

10.5 Consensus Package Schema

Field	Type	Description
baseline_claims	ClassifiedClaim[]	State-1 (accepted) claims with convergence scores
paraconsistent_claims	ParaClaim[]	State-2 claims with both-true contexts
rejected_claims	UUID[]	State-0 claim IDs (below threshold)
emergent_insights	Insight[]	Higher-order patterns (loops, transitions, bridges)
connection_graph	Graph	Full typed-relation graph across all claims
metadata	SessionMeta	Processing stats, agent counts, timing

10.6 Delivery Package Schema

Field	Type	Description
format	enum	Output format (JSON Markdown HTML PDF)
audience	AudienceSpec	Target audience profile
sections	Section[]	Ordered content sections with type tags
visualizations	Viz[]?	Optional charts, graphs, diagrams
source_map	Map	Traceability from output to input claims

11. Processing Loops and Feedback Mechanisms

The pipeline supports four classes of processing loops, each using Type B (feedback) connectors:

Loop Class	Scope	Trigger	Termination Condition	Max Iterations
Iterative Refinement	Within a single stage	Quality score below threshold on stage output	Quality score $\geq$ threshold OR max iterations reached	Configurable per stage (default: 3)
Cross-Stage Feedback	Stage N $\rightarrow$ Stage N-K	Consolidation discovers insufficient coverage in a matrix region	Coverage gap filled OR agent budget exhausted	Configurable (default: 2)
Longitudinal Learning	Session N $\rightarrow$ Session N+1	New session starts with same or related query domain	Explicit session close OR time-based expiry	Unlimited (bounded by storage)
Validation Loop	Post-Stage 4, pre-Stage 5	Consensus package fails consistency checks	All checks pass OR manual override	Configurable (default: 2)

Table 10: Four processing loop classes. All loops are bounded to prevent infinite cycling.

Loops interact with the fork system: a fork can internally implement any loop class, and loop triggers can activate forks (e.g., a cross-stage feedback loop discovering insufficient coverage can trigger a BP-2 fork to deploy additional specialist agents). The key constraint is that all loops must have a bounded termination condition—either a quality threshold, a maximum iteration count, or a resource budget limit.

12. Failure Modes and Mitigation

Failure Mode	Cause	Detection	Mitigation
Premature Consensus	Agents converge too quickly due to shared biases or overlapping training data	Low State-2 count ($\leq 10\%$ of claims) with high State-1	Increase agent diversity. Add adversarial agent with contrarian scope.
Infinite Contradiction	No claims reach State-1. All claims stuck in State-2 or State-0.	Zero baseline claims after consolidation	Lower convergence threshold. Increase agent count. Trigger cross-stage feedback.
Matrix Explosion	Too many exploration axes produce intractable cell count	Cell count exceeds agent budget by $>10\times$	Prune low-priority axes. Reduce values per axis. Use hierarchical matrix.
Echo Chamber	Agents cite same sources and reach same conclusions via different paths	Source overlap $>80\%$ across agent briefs	Diversify data source assignment. Enforce minimum unique sources.
Orphan Claims	Claims that connect to no other claims in the relation graph	Graph degree = 0 for claim nodes	Lower similarity threshold for connection. Manual review queue.
Fork Divergence	Fork outputs are incompatible at reconvergence point	Type mismatch at reconvergence	Registry validates type compatibility at registration time.
Stale Knowledge	Longitudinal connections reference outdated claims	Claim age exceeds configurable TTL	Deprecation markers. Re-validation loop on aged claims.
Agent Timeout	Agent exceeds timeline constraint without producing output	Wall-clock exceeds timeline_hours	Graceful degradation: use partial output. Reassign to new agent.

Table 11: Eight failure modes with detection signals and mitigation strategies.

13. Scaling Properties

13.1 Linear Scaling

Stage 3 (agent deployment) scales linearly with agent count when using P-thread execution—doubling agents doubles throughput given sufficient compute resources. Stage 5 (output synthesis) scales linearly with audience count. Both stages are embarrassingly parallel and benefit directly from additional compute.

13.2 Non-Linear Scaling

Stage 4 (consolidation) exhibits non-linear scaling. The number of pairwise claim comparisons grows as $O(n^2)$ where n is the total claim count. For k agents each producing c claims, this is $O((kc)^2)$. The connection graph traversal for emergent pattern detection can be $O(n^3)$ in the worst case. Mitigation strategies include: pre-filtering claims by embedding similarity before pairwise comparison (reducing effective n), hierarchical clustering to group related claims before cross-group comparison, and bounded graph traversal depth for pattern detection.

13.3 Scaling by Environment

Environment	Stage 3 Ceiling	Stage 4 Ceiling	Practical Agent Range
Single Node	CPU cores $\times$ 2 (I/O-bound agents)	RAM for claim vector index	5–20 agents
Cloud	Budget-limited. Effectively unlimited with auto-scale.	Managed vector DB scaling limits	20–200 agents
P2P Network	Peer count $\times$ peer capability	Distributed index convergence time	Network-dependent. 10–100 typical.

Table 12: Scaling ceilings by deployment environment.

14. Reference Implementation Guidance

This section provides guidance for teams implementing the architecture. The pipeline can be built incrementally—each phase adds capability while maintaining a functioning system:

Phase 1: Minimal Viable Pipeline (1–2 weeks)

Implement Stages 1, 3, and 5 only. Use a single LLM call for decomposition (Stage 1), skip matrix generation (Stage 2), deploy 3–5 agents with hardcoded constraint specs (Stage 3), skip consolidation (Stage 4), and concatenate agent outputs (Stage 5). This produces a basic multi-perspective report and validates the connector interfaces. Thread pattern: C-thread with T0 sub-threads.

Phase 2: Matrix + Consolidation (2–3 weeks)

Add Stage 2 (matrix generation) and Stage 4 (consolidation). Implement the morphological matrix generator with a simple scoring function. Implement baseline extraction (Sub-Stage A) for consolidation. This produces structured, consensus-based output. Thread pattern: C-thread with P-thread for Stage 3.

Phase 3: Paraconsistent Engine (2–4 weeks)

Add Sub-Stages B and C to consolidation. Implement State-2 classification and emergent pattern detection. Add Type D (graph-relational) connectors. This is the core intellectual contribution—the system now preserves and exploits contradictions. Thread pattern: F-thread for Stage 4 consolidation.

Phase 4: Full Modularity (2–4 weeks)

Implement the Fork Registry, branching points, and environment adapters. Add feedback loops and longitudinal connectors. Write the first domain-specific plug-in. At this point the system is fully modular and new capabilities can be added without modifying existing code.

15. Conclusion

This paper has presented a modular, domain-agnostic architecture for multi-agent knowledge synthesis that treats contradiction as a productive signal. The key contributions are:

- **A five-stage pipeline spine** with typed input/output contracts at every stage boundary, enabling component replacement without architectural changes.
- **Four connector types** (sequential, feedback, longitudinal, graph-relational) that formalize every possible inter-component interaction pattern.
- **Seven thread execution patterns** that compose hierarchically, supporting everything from minimal atomic execution to fully autonomous operation.
- **A three-valued paraconsistent consensus engine** that classifies claims as accepted (State 1), rejected (State 0), or paraconsistent (State 2), preserving contradictions as first-class outputs.
- **A spine+fork topology with four branching points** and a declarative fork registry, enabling snap-in modularity where new capabilities are added by implementing an interface and registering—never by modifying existing code.
- **Four environment-agnostic deployment abstractions** (compute, state, vector, message) that allow the same pipeline to run on a laptop, cloud cluster, or peer-to-peer network.

The architecture is intentionally domain-free. The structural patterns—decomposition, parallel exploration, paraconsistent consolidation, typed connections, and audience-adaptive synthesis—apply wherever complex knowledge work requires holding multiple perspectives simultaneously. A team implementing this for scientific literature search, competitive intelligence, legal analysis, medical diagnosis support, or engineering design space exploration would share the same Pipeline Spine and Execution Layer, differing only in their Domain Layer plug-ins.

The most profound insight embedded in this architecture is that **the disagreements between expert perspectives are often more valuable than their agreements**. By formalizing contradiction as State 2 rather than discarding it, the system preserves the richest signal in multi-agent synthesis—the signal that conventional consensus mechanisms destroy.